Strings

Strings are sequences of characters. Strings can contain alphabets, numbers and some special characters.

Strings can be created by enclosing characters inside a single quote or double-quotes.

Example:

S= ‘hello’

a= “python programming”

Multiple lines can be done using python’s triple quotes. It is also used for long comments in code. Special characters like TABs, NEWLINEs can also be used within the triple quotes. We can use either triple single or double quotes.

Example:

```
str='''python is an interpreted language,

python is dynamic typed language,

python is user friendly language'''

print(str)
```

output:

```
python is an interpreted language,

python is dynamic typed language,

python is user friendly language
```

Examples:

```
my_string = 'Hello'

print(my_string)
```

```
my_string = "Hello"
print(my_string)


my_string = '''Hello'''
print(my_string)

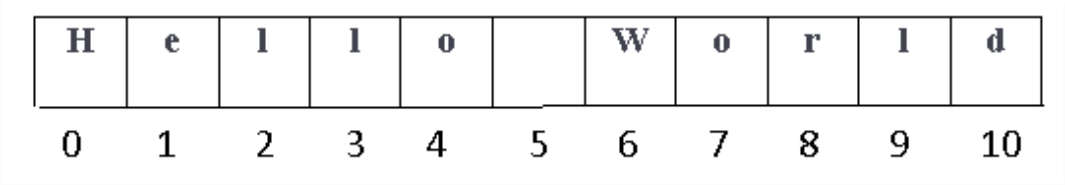

# triple quotes string can extend multiple lines
my_string = """Hello, welcome to
        the world of Python"""
print(my_string)
```

When you run the program, the output will be:

```
Hello
Hello
Hello
Hello, welcome to
        the world of Python
```

Accessing characters in Python:

- We can access the individual characters in a string through *indexing*.
- The positions in a string are numbered from the left, starting with 0.



>>>txt= ‘Hello World’

>>>txt[6]

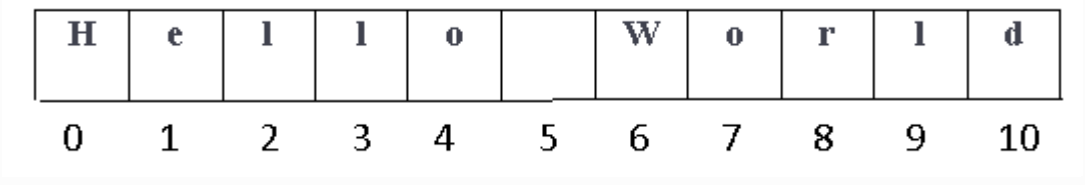
‘W’

- In a string of *n* characters, the last character is at position *n-1* since we start counting with 0.

String Slicing:

We can also access a contiguous sequence of characters, called a *substring*, through a process called *slicing*

- Slicing:
  <string>[<start>:<end>]
- start and end should both be integers
- The slice contains the substring beginning at position start and runs up to but doesn’t include the position end.



>>>s[2:7] # starting at 2, up to but not including 7

llo W

>>>s[6:10]

Worl

Slice from start: By leaving out the start index the range will start at the first character

>>>s[:5]

Hello

>>>s[:7]

Hello W

Slice to end: By leaving out the *end* index, the range will go to the end

>>>s[6:]

World

>>>s[2:]

llo World

Negative Indexing:

- As an alternative, Python supports using negative numbers to index into a string: -1 means the last char, -2 is the next to last, and so on. **We can index from the right side using negative indexes**

H	e	l	l	o		W	o	r	l	d
-11	**-10**	**-9**	**-8**	**-7**	**-6**	**-5**	**-4**	**-3**	**-2**	**-1**

>>>txt= ‘Hello World’

>>>txt[-4]

‘o’

>>>txt[-5]

‘W’

>>>txt[-5:-1]

Worl

>>>txt[-5:]

World

>>>txt[:-1]

Hello Worl

>>>txt[-5:-2]

Wor

>>>txt[:-4] #slice from start to -3(-4 not included)

Hello W

>>>txt[-5:] #begin slice with -5 to the end

World

#Accessing string characters in Python

str = 'GITAM University'

print('str = ', str)

```
#first character
print('str[0] = ', str[0])

#last character
print('str[-1] = ', str[-1])

#slicing 2nd to 5th character
print('str[1:5] = ', str[1:5])

#slicing 6th to 2nd last character
print('str[5:-2] = ', str[5:-2])
```

When we run the above program, we get the following output:

```
str =  GITAM University
str[0] =  G
str[-1] =  y
str[1:5] =  ITAM
str[5:-2] =  Universi
```

Modify/Delete a String in Python

Strings are immutable. It suggests that once a String binds to a variable, it can't be modified. In other words, This means that elements of a string cannot be changed once they have been assigned. We can simply reassign different strings to the same name.If you want to update the String, then re-assign a new String value to the same variable.

```
>>> my_string = 'GITAM'

>>> my_string[3] = 'a'

...

TypeError: 'str' object does not support item assignment

>>> my_string = 'Python'

>>> my_string

'Python'
```

Similarly, we cannot modify the Strings by deleting some characters from it. Instead, we can remove the Strings altogether by using the ‘del’ command.

```
str1 = "Python"

del str1[1]

# TypeError: 'str' object doesn't support item deletion

del str1

print(str1)

# NameError: name ‘str1' is not defined
```

Operations on strings

Assume string variable **a** holds 'Gitam' and variable **b** holds 'University', then −

Operator	Description	Example
+	Concatenation - Adds values on either side of the operator	a + b will give GitamUniversity

*	Repetition - Creates new strings, concatenating multiple copies of the same string	a*2 will give - GitamGitam
[]	Slice - Gives the character from the given index	a[1] will give i
[:]	Range Slice - Gives the characters from the given range	a[1:4] will give ita
in	Membership - Returns true if a character exists in the given string	‘t’ in a will give True
not in	Membership - Returns true if a character does not exist in the given string	‘B’ not in a will give True

Examples:

Concatenation of Two or More Strings: Joining of two or more strings into a single one is called concatenation. The + operator does this in Python. Simply writing two string literals together also concatenates them.

```
str1 = 'Hello'

str2 ='World!'

# using +

print('str1 + str2 = ', str1 + str2)
```

When we run the above program, we get the following output:

```
str1 + str2 =  HelloWorld!
```

Writing two string literals together also concatenates them like + operator.

If we want to concatenate strings in different lines, we can use parentheses.

```
>>> # two string literals together

>>> 'Hello "World!'

'Hello World!'
```

```
>>> # using parentheses

>>> s = ('Hello '

...      'World')

>>> s='Hello World'
```

2.Repetition : The ***** operator can be used to repeat the string for a given number of times.

```
str1 = 'Hello'

# using *

print('str1 * 3 =', str1 * 3)
```

When we run the above program, we get the following output:

```
str1 * 3 = HelloHelloHello
```

String Methods:

Strings are amongst the most popular types in Python. We can create them simply by enclosing characters in quotes. Python treats single quotes the same as double quotes.

1. capitalize()- capitalize first letter in the string

```
str='hello'
print(str.capitalize())
```

Hello

2. len()- Returns the length of string

```
str='hello'
print(len(str))
```

5

3. count(str) - return no of times str occurs in a string

```
str='hello'
print(str.count('l'))
```

2

4. upper()- converts all characters in the string into uppercase

```
str='Hello'
print(str.upper())
```

HELLO

5. lower()- converts all characters in the string into lowercase

```
str='Hello'
print(str.lower())
hello
```

6. swapcase()- upper case character becomes lowercase and viceversa

```
str='Hello'
print(str.swapcase())
```

hELLO

7. endswith()- returns True if the string ends with the specified value, otherwise False.

```
txt = "Hello, welcome to my world."
x = txt.endswith("my world.")
print(x)
```

True

8. startswith() - returns True if the string starts with the specified value, otherwise False.

```
txt = "Hello, welcome to my world."
x = txt.startswith("Hello.")
print(x)
```

False

9. find()- finds the first occurrence of the specified value and returns -1 if the value is not found.

```
txt = "Hello, welcome to my world."
x = txt.find("welcome")
print(x)
```

7

10. index()- finds the first occurrence of the specified value and raises an exception if the value is not found.

```
txt = "Hello, welcome to my world."
print(txt.find("q"))
print(txt.index("q"))
```

Output:-1
ValueError: substring not found

11. replace() - replaces a specified phrase with another specified phrase.

```
txt = "one plus one is two and one multipy one is one"
x = txt.replace("one", "three")
print(x)
```

output: three plus three is two and three multipy three is three

```
txt = "one plus one is two and one multipy one is one"
x = txt.replace("one", "three",2)
print(x)
```

Output: three plus three is two and one multipy one is one

12. rfind()-finds the last occurrence of the specified value, returns -1 if the value is not found.

```
txt = "Hello, welcome to my world."
x = txt.rfind("e")
print(x)
```

output:13

13. islower()- returns True if all the characters are in lower case, otherwise False.

```
txt = "hello world!"
x = txt.islower()
print(x)
```

True

14. isupper() - returns True if all the characters are in upper case, otherwise False.

```
txt = "hello world!"
x = txt.isupper()
print(x)
```

False

15. isdigit() - returns True if all the characters are digits, otherwise False.

```
txt = "50800"
x = txt.isdigit()
print(x)
```

True

16.isalpha()- Returns true if string has at least 1 character and all characters are alphabetic and false otherwise.

```
str = "Hello!welcome"
result=str.isalpha()
```

True

17.isalnum() -Returns true if string has at least 1 character and all characters are alphanumeric and false otherwise.

```
str = "Hello!welcome"
result=str.isalnum()
```

True

18. **title()-** This method returns a copy of the string in which first characters of a ll the words are capitalized.

```
str = "this is string example....wow!!!"
print(str.title())
```

output: This Is String Example....Wow!!!

19.**swapcase()** – This method is used to swap the case of all the case-based cha racters present in a string. That is, the lowercase characters in the string will be converted into uppercase characters and vice-versa.

```
str = "This is string example....WOW!!!"
print(str.swapcase())
```

output: tHIS IS STRING EXAMPLE....wow!!!

20. **strip()** – remove all the specified leading and trailing characters from the be ginning and the end of a string

```
str = "8888this is string example....wow!!!888"
print(str.lstrip('8'))
```

output:
  this is string example....wow!!!

21. **lstrip()** – strips all the specified characters in a string from the beginning.

```
str = "8888this is string example....wow!!!888"
print(str.lstrip('8'))
```

output:
  this is string example....wow!!!888

22.**rstrip()** – remove all the specified characters in a string from the right (i.e. e nding).

```
str = "8888this is string example....wow!!!888"
print(str.lstrip('8'))
```

output:
  8888this is string example....wow!!!

23. **max(str) -** Returns the max alphabetical character from the string str.

```
    str = "this is really a string example"
  print(max(str))
```

output: y

24. **min(str) -** Returns the min alphabetical character from the string str.

```
str = "example"
print(min(str))
output: a
```